



SECURITY AUDIT

Lista Dao

V3 Collateral

FINAL REPORT

July 2026

BAILSEC.IO



Disclaimer:

BailSec was engaged by the client to perform a time-boxed technical security assessment of the target identified in this report. The assessment applies only to the scope, version, commit, deployment, materials and information identified in the report. Its findings are point-in-time and not exhaustive. Changes made after the assessment—including remediation changes—were not assessed unless expressly stated otherwise and may affect the applicability of the report's conclusions.

No security assessment can identify every vulnerability or guarantee that a system is secure, error-free or protected against attack or loss. Only the components, assumptions, threat models and risk categories expressly identified as in scope were assessed. The report should be read in full, including its scope, exclusions, unresolved findings and remediation status. The client remains responsible for design, testing, remediation, deployment, operation, monitoring and incident response.

This report is a technical assessment, not a certification, attestation, investment recommendation, or legal, financial, regulatory or tax advice. It does not constitute an endorsement of the client, the assessed system or any related project.

This report was prepared for the client under a separate engagement agreement. If published, its public availability is for transparency and informational purposes only. To the fullest extent permitted by applicable law, publication does not create an auditor-client relationship, duty of care or right of reliance in favor of any third party. Other readers should conduct their own diligence and obtain appropriate professional advice.

Only the complete, unmodified report as issued by BailSec is authoritative. The signed engagement agreement exclusively governs the contractual rights and obligations between BailSec and the client, including warranties, remedies, liability limitations, indemnities and dispute resolution. Nothing in this notice excludes liability that cannot lawfully be excluded.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Lista Dao – V3 Collateral – Audit Report
Website	lista.org
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/lista-dao/moolah/tree/110544000c86e8bdf36971e7b7cdb3ec93a672f7
Resolution 1	https://github.com/lista-dao/moolah/tree/b607f2bebfac6d30a130d314f05d7837260caf1a

2. Detection Overview

Due to the high amount of H/M issues and leftover/unresolved issues after the resolution round, we recommend a follow-up audit before deployment.

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High	11	6		3	2	
Medium	9	3		5	1	
Low	29	14		12	1	2
Informational	9	5		4		
Governance	1			1		
Total	59	28		25	4	2

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

Global

Issue_01	0 amount approvals can revert for some tokens
Severity	Low
Description	Certain tokens like BNB on Ethereum mainnet reverts on 0 amount approval. <code>safeApprove</code> and <code>safeIncreaseAllowance</code> is called with 0 amount in multiple locations in the codebase eg: <code>V3Liquidator.liquidate()</code> , <code>mint</code> and <code>increaseLiquidity</code> functions in <code>V3PositionLib</code>
Recommendations	In the event that tokens having this behavior are used in conjunction with these contracts, catch the revert
Comments / Resolution	Acknowledged

Issue_02	DEFAULT_ADMIN_ROLE can upgrade all UUPS contracts without a timelock
Severity	Governance
Description	The V3Provider, V3DexAdapter, V3Liquidator, V3ProviderOracle, and SlisBNBxMinter contracts all use the UUPS upgrade pattern, with _authorizeUpgrade gated by DEFAULT_ADMIN_ROLE. A holder of this role can replace the implementation of any of these contracts in a single transaction with no delay. This gives the admin unilateral, immediate control over all contract logic, including share accounting, oracle pricing, liquidation flows, and fund custody. If the DEFAULT_ADMIN_ROLE is held by an EOA or an ungoverned multisig, a key compromise allows an attacker to deploy a malicious implementation that drains all funds across the entire V3 provider system in one transaction. Even without compromise, the absence of a timelock means users have no observation window to react to an upgrade before it takes effect.
Recommendations	Consider assigning DEFAULT_ADMIN_ROLE to a multisig wallet behind a timelock contract so that upgrades are subject to a mandatory delay, giving users and monitoring systems time to review and respond before new implementations take effect.
Comments / Resolution	Acknowledged

V3Provider

The V3Provider contract is an abstract two-token vault which converts ownership of a concentrated-liquidity position into fungible provider shares. These shares are used as the collateral token inside a corresponding Moolah market.

Users deposit the two pool tokens, or native currency for the wrapped-native leg, after which the assets are forwarded to the configured **V3DexAdapter** and added to the managed V3 position. Before issuing new shares, the contract collects and compounds existing fees so that previously accrued value remains attributable to existing shareholders. The amount of shares minted is based on the fair value of the liquidity actually added relative to the value and share supply of the existing position.

The contract allows users to withdraw the provider shares themselves or redeem them for their proportional part of the underlying liquidity and idle token inventory. The standard single-token ERC-4626 deposit and withdrawal functions are disabled because the strategy requires two-token liquidity.

Appendix: Deposit and Share-Minting Flow

The provider uses a two-token deposit flow rather than the standard single-asset ERC-4626 mechanism. A deposit is processed as follows:

1. The contract verifies that the selected **Moolah** market uses the provider share as its collateral token and that **onBehalf** is a valid address.
2. If native currency is supplied, it is wrapped into the configured wrapped-native token and assigned to the corresponding pool leg; all remaining ERC-20 inputs are transferred from the depositor.
3. **collectAndCompound** is invoked before the new deposit is valued so that previously accrued fees and tracked idle inventory remain attributable to the existing shareholders.
4. The provider records the existing share supply and values the managed position at **ADAPTER.fairSqrtPriceX96[]**
5. The two input tokens are forwarded to the adapter and added to the active V3 position; any amount that cannot be consumed because of the pool ratio is returned to the provider.
6. The liquidity actually minted is decomposed at the fair valuation price and valued through the resilient oracle.

For an existing vault, the share amount is calculated as:

```
> shares = addedValueUsd * supplyBefore / totalValueBefore
```




For the first deposit, the fair USD value is converted into the configured accounting-asset denomination:

> $\text{shares} = \text{addedValueUsd} * 10^{\text{accountingAssetDecimals}} / \text{accountingAssetPrice}$

The newly minted shares are first issued to the provider, approved to **Moolah**, and supplied as collateral for `onBehalf`. Only after the share supply and Moolah collateral accounting have been updated are the unused token amounts refunded to the original depositor.

Appendix: Withdrawal and Redemption Flows

The provider exposes three distinct exit paths:

- Underlying withdrawal: **Moolah** releases the requested collateral shares to the provider, the collateral-change hook is executed, fees are compounded, the shares are burned, and the adapter sends the corresponding pro-rata underlying tokens directly to the receiver.
- Share withdrawal: **Moolah** releases the provider shares and the shares are transferred to the receiver without being burned; the underlying position and **totalSupply** remain unchanged.
- Standalone redemption: A holder of provider shares burns them directly through **redeemShares** and receives the corresponding fraction of the adapter's liquidity and tracked idle inventory without interacting with a **Moolah** position.

When underlying assets are redeemed, the provider passes the pre-burn total supply to the adapter so that the removed fraction remains:

> $\text{redeemedFraction} = \text{sharesRedeemed} / \text{totalSupplyBeforeBurn}$

The wrapped-native leg is unwrapped by the adapter and delivered as native currency, while the other leg is transferred as an ERC-20 token.

Privileged Functions

- `_authorizeUpgrade`
- `transfer`
- `transferFrom`
- `liquidate`
- `receive`

Invariants

INV 1: `ADAPTER.provider()` must equal the provider's address and the provider's configured adapter must never be replaced by an unrelated adapter

INV 2: The provider's `TOKEN0`, `TOKEN1`, wrapped-native token, and token decimals must match those configured in its adapter

INV 3: `totalSupply()` must equal the sum of all provider-share balances

INV 4: Every provider share must represent the same pro-rata claim on the adapter's managed liquidity, fees, and tracked idle balances

INV 5: Existing fees and idle inventory must be collected or accounted for before calculating shares for a new deposit

INV 6: A deposit must mint shares only for the fair value of liquidity actually added and not for refunded or unused token amounts

INV 7: A share-only withdrawal must reduce `Moolah` collateral and increase the receiver's share balance without changing `totalSupply` or adapter liquidity

INV 8: A successful deposit must increase `totalSupply` and the beneficiary's `Moolah` collateral by exactly the same number of shares

Issue_03	Flash loan on provider-share collateral enables mint/redeem value arbitrage and zero-price grieving
Severity	High
Description	Moolah::flashLoan() allows the provider-share collateral token to be flash-borrowed. An attacker can perform a full redemption, driving the totalSupply to zero and then re-mint the same amount of shares by supplying less underlying assets. These shares can then be supplied to the Moolah contract making the flashLoan function execute successfully. The attacker profits from the difference between the fees-per-share accrued prior to the attack and the plain oracle-based valuation in the supplyBefore == 0 branch of deposit
Recommendations	Disable flash loans for the provider-share collateral token (and for any market whose collateral token is a V3 provider share)
Comments / Resolution	Acknowledged. Lista will disable flash loans for V3 Provider markets

Issue_04	ERC-4626 inflation attack surface on provider shares
Severity	High
Description	Although this vault is not a standard single-asset ERC-4626 (single-asset entry is disabled), the share/underlying accounting is still susceptible to the classic ERC-4626 first-depositor/inflation attack. An attacker can be the first depositor, mint a tiny share supply, withdraw to 1 wei share, and then donate/inflate the underlying position value via NPM's <code>increaseLiquidity</code> or the underlying pool's <code>flash</code> so that subsequent depositor's contributions round down to fewer shares, transferring a significant value to the attacker Eg: - Attacker mints 1e18 shares by adding liquidity worth 1 USD (1e8 in decimals) - Attacker withdraws 1e18 - 1 shares leaving behind approx 1e-8 USD - Attacker notices a victim about to deposit 1000 USD worth of liquidity - Attacker front runs the deposit by adding 501 USD worth of liquidity - Now since 1 wei share is worth > 501 USD, the victim will only receive 1 wei share. This results in approximately a loss of 250 USD for the victim and the same amount of gain for the attacker
Recommendations	Consider adding a <code>minShares</code> parameter to <code>deposit</code> so that the transaction reverts if the minted shares fall below the caller's specified slippage threshold. Alternatively, make use of the OpenZeppelin base contract's <code>_decimalsOffset</code> functionality to make such attacks economically non-viable
Comments / Resolution	Resolved in commit 0da856bd15dfd086b3bcf87bf0087651f515d747 along with operational solution of dead share seeding

Issue_05	<code>deposit</code> mints shares from the fair-price valuation of liquidity paid for at spot
Severity	High
Description	In <code>deposit</code>, the user's tokens are added to the position via <code>addLiquidity</code>, which mints liquidity at the pool spot price and consumes <code>amount0Used/amount1Used</code> accordingly. Shares, however, are not derived from the tokens actually paid – they are derived from <code>addedValue = _amountsValueUsd(added0, added1)</code>, where <code>[added0, added1] = amountsForLiquidity(liquidityAdded, fairSqrtPriceX96)</code> is the fair-price re-decomposition of the minted liquidity. When the spot price and the fair valuation price diverge, this decomposition is worth more (through the resilient oracle) than the valuation at spot. When there are idle tokens, this causes the ratio (newly added liquidity value/existing liquidity position value + idle token value) to be higher and the depositor is credited more shares than their contribution is worth. Because <code>withdraw/redeemShares</code> return pro-rata underlying, the over-credited depositor can then withdraw for a net profit at existing holders' expense Eg: $P_0 = 1e8$ $P_1 = 0.9e8$ hence oracle price = 1.111111111 $L_{Before} = 1e18$ $L_{Add} = 1e17$ spotPrice = 1.05126846837676659 upperTickPrice = 1.105165392603232697 lowerTickPrice = 0.904841941932768878 fairPrice = 1 idleAmount0 = $1e18$ idleAmount1 = $1e18$ totalShares = 199265956 total existing value at fair price composition = 1.99265956 (includes idle token amounts) adding the liquidity,

	input token0 amount = 0.002407932866649372 input token1 amount = 0.007408203654804451 real value added ie. at spot price composition = 0.00907531 added value at fair price composition = 0.00926594 obtained shares = 926594 on withdrawing, the user receives, token0 amount = 0.004751110556377053 token1 amount = 0.005005692607594631 received value = 0.00925623 profit = 0.00018092
Recommendations	There two independent viable fixes: • Use the oracle price for liquidity decomposition • Add deposits to idle capital without adding liquidity to the pool or consulting the oracle; require that deposits be balanced according to the total composition of the position (inclusive of accumulated fees)
Comments / Resolution	Unresolved. In case the fair price composition is valued less than the spot price composition, a deposit-withdraw cycle is still profitable for the depositor. Apart from that input token amounts should be rounded up rather than down favoring the existing shareholders.

Issue_06	<code>deposit</code> DoS when position value rounds to zero with non-zero supply
Severity	Medium
Description	<code>V3Provider::deposit()</code> reverts with <code>ZeroShares</code> when <code>supplyBefore > 0</code> but <code>totalValueBefore == 0</code>. This can DoS all deposits into the contract for as long as that condition holds. An attacker can drive the contract to this state by having a minimal amount of liquidity which rounds down to 0 valuation Eg: <code>price0 = price1 = 1e8</code> <code>tickLower = -100</code> <code>tickUpper = 100</code> <code>fairTick = 0</code> liquidity of 2005104164790 will have amounts of 9999999999 <code>token0</code> and <code>token1</code> when computing the value, this rounds down to 0
Recommendations	Maintain a small dust position in the pool, potentially by using an economically relevant amount of “dead” shares.
Comments / Resolution	Resolved operationally by seeding with dead shares

Issue_07	Transfer restrictions can be bypassed by <code>supplyShares-withdrawShares</code> combination
Severity	Low
Description	<code>V3Provider</code> has transfer restrictions kept which only allows Moolah to perform token transfers. But this can be bypassed using a combination of <code>supplyShares</code> and <code>withdrawShares</code> keeping the intended transfer recipient as the receiver inside <code>withdrawShares</code>
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged

Issue_08	<code>deposit</code> rounds existing value down, minting excess shares
Severity	Low
Description	In <code>deposit</code> , $\text{shares} = \lfloor \text{addedValue} * \text{supplyBefore} \rfloor / \text{totalValueBefore}$ where <code>totalValueBefore</code> is the existing position value. Because <code>totalValueBefore</code> is rounded down, the denominator is smaller than the true value, so the depositor is minted <i>slightly</i> more shares than their contribution warrants, diluting existing holders
Recommendations	Consider rounding <code>totalValueBefore</code> up
Comments / Resolution	Resolved in commit <code>af0bff2272a787b519d4c5ab2a84875f8f34c76e</code>

Issue_09	<code>withdraw</code> compounds after the health check, under-valuing collateral
Severity	Low
Description	<code>withdraw</code> calls <code>MOOLAH.withdrawCollateral</code> and <code>_afterCollateralChange</code> and then <code>collectAndCompound</code> after those two calls. Invoking <code>collectAndCompound</code> at this point moves fees/idle into liquidity and can lower the reported collateral value at the moment the Moolah health check and <code>slisBNBx</code> sync run, making the position look less healthy than it is (and mis-syncing <code>slisBNBx</code>)
Recommendations	Order the operations so that fees are compounded before the health-check-relevant valuation is taken. Alternatively, ensure the valuation used for the health check and <code>slisBNBx</code> sync consistently reflects the post-compound state
Comments / Resolution	Resolved in commit <code>f0d4499eb735c5ffffa97d5ceacb9285f91ae007</code>

Issue_10	<code>collectAndCompound</code> is not invoked inside multiple functions
Severity	Low
Description	Both <code>withdrawShares</code> and <code>supplyShares</code> don't invoke <code>collectAndCompound</code> before transferring the shares from Moolah. This will cause the health check to be performed without including the pending fees
Recommendations	Add an invocation of <code>collectAndCompound</code> to these functions before transferring shares to Moolah. Alternatively, include the uncollected fees in the health check function
Comments / Resolution	Resolved in commit <code>f0d4499eb735c5ffffa97d5ceacb9285f91ae007</code>

Issue_11	Provider valuation not fail-closed on zero leg
Severity	Low
Description	<code>_amountsValueUsd</code> computes <code>amount0*price0 + amount1*price1</code> with no <code>price == 0</code> guard, and <code>totalAssets</code> guards only <code>assetPrice == 0</code> , so <code>deposit</code> keeps computing on a one-legged valuation during a broken-feed window in which the oracle's <code>peek</code> hard-reverts <code>'ZeroPrice'</code>
Recommendations	Acknowledge this finding. Alternatively, revert in the case that either price is 0
Comments / Resolution	Resolved in commit <code>f0d4499eb735c5ffffa97d5ceacb9285f91ae007</code>

Issue_12	<code>transferFrom</code> skips allowance spending, leaving dangling approvals from <code>supplyShares</code>
Severity	Low
Description	In the <code>transferFrom</code> override, <code>_transfer[from, to, value]</code> is called directly, without invoking <code>_spendAllowance</code> . Hence granted allowances are never decremented by any transfer.
Recommendations	Call <code>_spendAllowance</code> within the <code>transferFrom</code> override
Comments / Resolution	Resolved in commit <code>f0d4499eb735c5ffffa97d5ceacb9285f91ae007</code>

Issue_13	ERC-4626 specification is not followed by V3Provider
Severity	Informational
Description	The standard mint,deposit,withdraw,redeem functions have been disabled in V3Provider. But multiple view functions doesn't take this into account violating multiple EIP-4626 specifications - maxDeposit,maxMint,maxWithdraw and maxRedeem return non-zero values instead of 0 - previewDeposit, previewMint, previewWithdraw, and previewRedeem return computed values instead of reverting - totalAssets is not the amount of assets that is actually managed by the vault but is rather a synthetic valuation of a two-token position - convertToShares and convertToAssets return values based on a synthetic single-asset USD valuation of a two-token position, which is misleading since the standard entry points that would use these conversions always revert. Additionally, the contract has custom Deposit and Withdraw events which don't follow the specification
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged

Issue_14	*Changed events can be emitted when new value is the same
Severity	Low
Description	Both <code>MaxRebalanceLossBpChanged</code> , <code>MaxDailyLossUsdChanged</code> events are emitted even when the new value set is the same as the existing value. The event name is unsuitable here since the value hasn't actually changed
Recommendations	Revert in-case the new value is same as the existing value
Comments / Resolution	

Issue_15	<code>previewDeposit*</code> returned amounts can differ due to lack of simulation of <code>collectAndCompound</code>
Severity	Low
Description	The <code>previewDepositAmounts</code> , <code>previewDepositForToken0</code> and <code>previewDepositForToken1</code> functions compute the token amounts when <code>totalSupply</code> is <code>> 0</code> by assuming that the total amount of tokens returned by <code>positionAmountsAt</code> is accurate. This is flawed as before the actual deposit <code>collectAndCompound</code> will be invoked which can deposit the pending fees and idle tokens as liquidity into the pool. Following which taking the fair price composition can result in completely different token amounts
Recommendations	Simulate <code>collectAndCompound</code> rather than relying on <code>positionAmountsAt</code>
Comments / Resolution	

SlisBNBV3Provider

The SlisBNBV3Provider contract is a slisBNB-specific extension of **V3Provider**. It inherits the base deposit, share issuance, withdrawal, and redemption mechanism while adding the accounting required to mirror users' Moolah collateral positions into the slisBNBx reward system.

The contract records each user's collateral balance per Moolah market through **userMarketDeposit** and aggregates those balances through **userTotalDeposit**. These values are synchronized after deposits, withdrawals, and liquidations, and may also be synchronized permissionlessly through the individual or bulk synchronization functions.

The contract exposes each user's proportional position value in **BNB**, using the adapter's fair token composition and the resilient oracle prices. Whenever the **slisBNBx** minter is configured, an affected user is rebalanced in the minter after their tracked collateral changes. The contract also exposes a **BOT**-controlled function which forwards position rebalances to the slisBNB adapter.

Appendix: slisBNBx Deposit Mirroring

The **slisBNB** provider extends the base collateral flow with a secondary accounting layer used by the **slisBNBx** minter. Each user's collateral is tracked both per **Moolah** market and in aggregate:

```
> userTotalDeposit[account] =  $\Sigma$  userMarketDeposit[account][marketId]
```

After a deposit, withdrawal, or **Moolah** liquidation callback, **_afterCollateralChange** invokes **_syncPosition**. The synchronization flow is:

1. Read the user's current collateral amount directly from **MOOLAH.position[id, account].collateral**.
2. Compare it with the previously stored **userMarketDeposit[account][id]**.
3. Apply only the increase or decrease to **userTotalDeposit[account]**.
4. Replace the stored market balance with the current **Moolah** balance.
5. If a **slisBNBx** minter is configured, invoke **rebalance(account)** after the updated total has been recorded.



`syncUserBalance` and `bulkSyncUserBalance` expose the same reconciliation process permissionlessly, while requiring every supplied market to use this provider as its collateral token.

Appendix: BNB-Denominated User Balance

`getUserBalanceInBnb` converts a user's tracked provider shares into a BNB-denominated balance for the `slisBNBx` reward mechanism. The contract first determines the user's proportional token amounts at the adapter's fair composition:

```
> userToken0 = totalToken0 * userShares / totalSupply  
> userToken1 = totalToken1 * userShares / totalSupply
```

Both legs are then valued through the resilient oracle and divided by the BNB/USD price:

```
> userBalanceInBnb = (userToken0Usd + userToken1Usd) / bnbUsdPrice
```

The function therefore reflects the user's proportional ownership of V3 liquidity, stored fees reported by the adapter, and tracked idle inventory rather than merely the nominal number of provider shares.

Privileged Functions

- `rebalance`
- `setSlisBNBxMinter`

Invariants

INV 1: `userMarketDeposit[account][id]` must equal `MOOLAH.position(id, account).collateral` for every synchronized `slisBNB` provider market

INV 2: `userTotalDeposit[account]` must equal the sum of `userMarketDeposit[account][id]` across all `slisBNB` provider markets

INV 3: A user with zero `userTotalDeposit` must always have a reported BNB-denominated balance of zero

INV 4: When the `slisBNBx` minter is configured, every effective user-deposit change must synchronize the user's reward balance using the updated total deposit

Issue_16	Unregistered provider-collateral markets let the same shares inflate slisBNBx accounting
Severity	High
Description	<code>V3Provider</code> accepts any market whose <code>collateralToken == address[this]</code>, even if <code>Moolah</code> has not registered this provider for that market id. In such an unregistered market, <code>supplyShares</code> still calls <code>Moolah.supplyCollateral</code> and <code>SlisBNBV3Provider._afterCollateralChange</code> syncs/mints slisBNBx from the resulting collateral balance. The user can then withdraw the same shares directly through <code>Moolah.withdrawCollateral</code>, and that direct withdrawal does not invoke the provider hook – leaving <code>userMarketDeposit/userTotalDeposit</code> stale. Repeating across fresh market ids lets the same shares inflate slisBNBx accounting and minting repeatedly
Recommendations	Require that the market is registered for this provider in every provider entrypoint (<code>deposit</code> , <code>supplyShares</code> , <code>withdrawShares</code> , <code>withdraw</code>), so shares can only enter markets where the provider hook governs both supply and withdrawal
Comments / Resolution	Acknowledged. Lista will ensure that every launched V3 LP market has its provider configured before activation; unsupported markets will not be created.

Issue_17	Price inconsistency across deposit and getUserBalanceInBnb for the same asset
Severity	Low
Description	The first deposit mints shares using the WBNB oracle price, while getUserBalanceInBnb converts the position value using the native BNB oracle price. At time of writing, WBNB is priced at 56883758500 while BNB is priced at 56879873331, a delta of 0.0068305%. The lower native BNB price causes users' balances to be overstated by roughly 0.0068305%, resulting in minor reward over-crediting. This does not affect Moolah collateral valuation or liquidations. However, the reward discrepancy would grow proportionally if the WBNB/BNB oracle spread widened
Recommendations	Maintain consistency across both deposit and getUserBalanceInBnb by using the same oracle price
Comments / Resolution	Resolved. The underlying resilient oracle configuration has been updated to return the same values for BNB and WBNB

Issue_18	<code>getUserBalanceInBnb</code> price updates require manual per-user sync
Severity	Low
Description	<code>getUserBalanceInBnb</code> values a user's shares based on the current liquidity position of the adapter, the fair price, the idle token amounts and the oracle prices, But the <code>slisBNBx</code> minter only observes a user's value when <code>_syncPosition</code> → <code>rebalance(account)</code> is triggered for that specific user. The actual value can change due to changes to any of the above, causing every user's entitlement to be stale until each is individually synced (<code>syncUserBalance/bulkSyncUserBalance</code>), so <code>slisBNBx</code> balances drift from the intended value between syncs
Recommendations	Acknowledge this finding. Document and operationalize a keeper that periodically bulk-syncs users' <code>slisBNBx</code> balances, or on-demand in the case of any major value change
Comments / Resolution	Acknowledged. Lista will operate a bot to synchronize balances

Issue_19	<code>_syncPosition</code> can be DoSed when the MPC wallet cap is reached
Severity	Low
Description	<code>_syncPosition</code> calls <code>ISlisBNBxMinter.rebalance(account)</code> after every position change, which routes fee <code>slisBNBx</code> through the MPC wallets. When the MPC wallet cap is reached, minting to the MPC wallets can revert, which in turn reverts the sync and therefore the deposit, withdrawal, or liquidation that triggered it – DoSing those flows
Recommendations	Cap fee minting gracefully rather than reverting, or ensure sufficient MPC wallet capacity is always provisioned and monitored. Liquidations in particular must never be blocked by fee-side reverts
Comments / Resolution	Acknowledged

Issue_20	Zero leg-price transiently burns reward balance
Severity	Low
Description	<code>SlisBNBV3Provider.getUserBalanceInBnb</code> uses <code>price0/price1</code> without the <code>ZeroPrice</code> guards <code>V3ProviderOracle.peek</code> applies, so a transiently-zero leg price yields an understated (possibly 0) BNB value instead of reverting, and <code>minter.rebalance</code> then burns the user's <code>slisBNBx</code> toward 0
Recommendations	Acknowledge this finding. Alternatively, revert in case the returned price is 0
Comments / Resolution	Resolved in commit <code>f0d4499eb735c5fffa97d5ceacb9285f91ae007</code>

WbETHV3Provider

The WbETHV3Provider contract is the wbETH/WETH implementation of **V3Provider** on Ethereum. It inherits the complete two-token deposit, Moolah collateral supply, share withdrawal, and underlying redemption mechanism from the base provider.

Unlike the slisBNB implementation, this contract does not maintain additional per-user reward accounting or mirror deposits into a secondary reward token. Its only additional operational function is a BOT-controlled rebalance entry point which forwards the supplied execution parameters to the configured **WbETHV3DexAdapter**

Appendix: wbETH/WETH Rebalance Forwarding

The **wbETH** provider does not execute pool or NFT operations itself. Rebalancing is initiated by an account holding the **BOT** role and forwarded to the configured adapter through the following flow:

1. The **BOT** supplies the minimum token-removal amounts, minimum replacement liquidity, execution deadline, and backend-generated swap calldata.
2. The provider applies its non-reentrancy and **BOT**-role checks.
3. The parameters are forwarded unchanged to **WbETHV3DexAdapter.rebalance**.
4. The adapter independently enforces its **onlyProvider** restriction, deadline, exchange-rate deviation threshold, swap-venue whitelist, and minimum-liquidity requirement.

The rebalance does not mint or burn provider shares; it only changes the range and token composition of the assets represented by the existing share supply.

Privileged Functions

- rebalance

Invariants

INV 1: Deposits, withdrawals, share issuance, and redemptions must obey the base **V3Provider** pro-rata accounting invariants

INV 2: The provider's adapter must be the configured **WbETHV3DexAdapter** and must identify this provider as its sole provider

INV 3: Only an account holding the provider's **BOT** role may initiate a **wbETH/WETH** strategy rebalance

INV 4: Rebalance forwarding must not allow the **BOT** to bypass the adapter's provider-only authorization

INV 5: The provider must use **wbETH** as **TOKEN0**, **WETH** as **TOKEN1**, and **WETH** as its wrapped-native token

No issues found.

WstETHV3Provider

The WstETHV3Provider contract is the wstETH/WETH implementation of [V3Provider](#) on Ethereum. It uses the base provider to represent the managed wstETH/WETH concentrated-liquidity position through fungible shares which can be supplied as collateral to Moolah.

The contract does not introduce additional user-deposit or reward-token accounting and therefore inherits the base deposit, withdrawal, redemption, and valuation behavior unchanged. It only adds a BOT-controlled rebalance function which forwards the requested range and inventory-conversion operation to the configured [WstETHV3DexAdapter](#)

Appendix: wstETH/WETH Rebalance Forwarding

The [wstETH](#) provider is a thin strategy-control layer over the base provider. A rebalance is performed as follows:

1. An account holding the [BOT](#) role submits the minimum token-removal amounts, minimum replacement liquidity, deadline, and optional inventory-conversion calldata.
2. The provider verifies the role and enters the non-reentrant execution context.
3. All rebalance parameters are forwarded unchanged to [WstETHV3DexAdapter](#).
4. The adapter validates that the call originated from its configured provider and performs the complete NFT removal, inventory conversion, range recentering, and replacement mint.

Because the operation is performed entirely against assets already owned by the strategy, a rebalance must not alter the provider's total share supply or any user's nominal Moolah collateral-share balance.

Privileged Functions

- rebalance

Invariants

INV 1: The provider must use `wstETH` as `TOKEN0`, `WETH` as `TOKEN1`, and `WETH` as its wrapped-native token

INV 2: Deposits, withdrawals, share issuance, and redemptions must obey the base `V3Provider` pro-rata accounting invariants

INV 3: The provider's adapter must be the configured `WstETHV3DexAdapter` and must identify this provider as its sole provider

INV 4: Only an account holding the provider's `BOT` role may initiate a `wstETH/WETH` strategy rebalance

INV 5: Rebalance forwarding must not allow the `BOT` to bypass the adapter's provider-only authorization

No issues found.

V3DexAdapter

The V3DexAdapter contract is an abstract DEX adapter which holds and manages a single Uniswap V3 or PancakeSwap V3 concentrated-liquidity NFT on behalf of a **V3Provider**. It separates NFT custody and pool interaction from the provider's share and Moolah accounting.

Only the configured provider can add liquidity, remove liquidity, compound fees, or rebalance the position. The adapter tracks the active NFT, its tick range, and any idle token inventory which could not be placed into the position because of the required token ratio. Idle balances are tracked explicitly rather than through the adapter's raw token balances so that unsolicited token transfers do not automatically increase the reported position value.

During a rebalance, the adapter removes and collects the old position, optionally converts inventory through a whitelisted swap venue, calculates a new rate-centered range, and mints a replacement NFT. The adapter also handles wrapped-native token conversions, returns unused deposit amounts, and exposes spot, TWAP, and fair-price composition views used by the provider and its oracle.

Appendix: Position Lifecycle

The adapter is the sole custodian of the strategy's V3 NFT and manages at most one active **tokenId**.

- On the first liquidity addition, the adapter mints a new NFT using the configured token pair, fee tier, and tick range.
- On later deposits, liquidity is increased on the existing NFT.
- Any part of the desired token amounts that cannot be consumed by the V3 position is returned to the provider; wrapped-native refunds are unwrapped and returned as native currency.
- **positionAmountsAt** reports the principal represented by the NFT at the selected price, the NFT's stored **tokensOwed**, and the explicitly tracked **idleToken0** and **idleToken1**.
- During redemption, the adapter removes the share holder's proportional liquidity and the same proportional fraction of both idle balances before transferring the resulting assets to the receiver.

The adapter records idle balances in storage rather than using its full ERC-20 balances so that unsolicited token donations do not automatically become part of the provider-share NAV.

Appendix: Fee Collection and Compounding

`collectAndCompound` collects every token amount currently owed to the NFT and combines it with the existing idle inventory:

> `toCompound0 = collected0 + idleToken0`

> `toCompound1 = collected1 + idleToken1`

The combined amounts are supplied to `increaseLiquidity` on the active NFT. Any ratio-mismatched remainder is stored again as idle inventory:

> `idleToken0After = toCompound0 - amount0Used`

> `idleToken1After = toCompound1 - amount1Used`

Compounding does not change the provider's share supply; it only converts fees and idle inventory into additional liquidity represented by the existing shares.

Appendix: Rebalance Flow

A rebalance replaces the current NFT with a position centered around the latest LST/native exchange rate, or around a fallback pool tick when no rate is available. The operation follows this sequence:

1. Verify the deadline and, for rate-implied pairs, require sufficient movement from `lastCenterRate`.
2. Derive the new tick range; rate-implied ranges are centered around the current exchange rate with an approximate $\pm 1\%$ width and aligned to the pool tick spacing.
3. Collect existing owed tokens and combine them with all tracked idle inventory.
4. Remove the full liquidity of the old NFT, collect the resulting tokens, burn the empty NFT, and clear `tokenId`.
5. Optionally convert one inventory leg through a manager-whitelisted swap venue using backend-generated calldata.
6. Store the new tick range and mint one replacement NFT with the resulting inventory.
7. Record every token amount not consumed by the replacement mint as idle inventory.
8. Enforce `minLiquidity` and, for rate-implied pairs, update `lastCenterRate`.

The entire operation is atomic: if any deadline, slippage, whitelist, rate-deviation, or minimum-liquidity check fails, the previous position and accounting state are restored by the transaction revert.

Privileged Functions

- `_authorizeUpgrade`
- `setProvider`
- `addLiquidity`
- `removeLiquidity`
- `collectAndCompound`
- `rebalance`
- `setCenterRateThresholdBps`
- `setSwapPairWhitelist`
- `receive`

Invariants

INV 1: Only the configured provider may call `addLiquidity`, `removeLiquidity`, `collectAndCompound`, or `rebalance`

INV 2: The adapter must manage at most one active Uniswap V3 NFT position

INV 3: When an active NFT exists, the adapter must own it and its token pair, fee tier, and tick range must match the adapter configuration

INV 4: `tickLower` must remain below `tickUpper` and both ticks must be valid multiples of the pool's tick spacing

INV 5: `idleToken0` and `idleToken1` must never exceed the adapter's spendable balances of the corresponding tokens

INV 6: For each deposit leg, the tokens transferred to the adapter must equal the tokens added to liquidity plus the tokens refunded

INV 7: Any replacement-mint tokens that are not consumed as liquidity must be recorded as idle inventory

INV 8: A rebalance venue must not be able to spend more than the amount explicitly approved or sent for the current swap.

INV 9: When the center-rate threshold is active, rebalance must occur only after the required deviation from `lastCenterRate`

Issue_21	<code>positionAmountsAt</code> ignores accrued fees, corrupting every downstream valuation
Severity	High
Description	<code>positionAmountsAt</code> computes token amounts from <code>liquidity</code> and adds <code>tokensOwed{0,1} + idleToken{0,1}</code> , but <code>tokensOwed</code> only reflects fees already <i>checkpointed</i> into the position; fees accrued since the last NPM interaction are not included. Because this view underpins <code>totalAssets</code> , <code>_positionValueUsd</code> , <code>getUserBalanceInBnb</code> , <code>getTotalAmounts</code> , and the oracle <code>peek</code> , every valuation is systematically understated by the pending fee amount. Effects include: incorrect collateral pricing for health checks and liquidations, and incorrect slisBNBx entitlements.
Recommendations	Account for pending fees by simulating fee growth (<code>feeGrowthInside</code> deltas) so that uncollected fees are included in <code>positionAmountsAt</code>
Comments / Resolution	Resolved. Now the pending fees is included in the returned amount

Issue_22	Pre-rebalance liquidity addition bypasses the <code>minLiquidity</code> guard
Severity	High
Description	<code>rebalance</code> only enforces <code>minLiquidity</code> against the newly-minted liquidity after conversion. An attacker can add liquidity to the position (or manipulate inventory) and manipulate the price immediately before the rebalance so that the <code>minLiquidity</code> check passes on inflated liquidity, while the liquidity was actually supplied at an off-market price. The rebalanced liquidity can then be swapped against for profit
Recommendations	Add an explicit check for deviation between a target price pass as an argument to <code>rebalance</code> and actual pool price at the end of the swapping and liquidity adding operations.
Comments / Resolution	Resolved. Rebalance is now aborted in case spot price deviates from the target price by greater than allowed deviation. The value set for max deviation will determine the extent to which the manipulation can occur Note that the roundings involved for the acceptable price range computation doesn't always round for a more strict range, eg: lower end is rounded down rather than up, <code>priceB</code> is rounded down even for the lower end computation. It is ideal that these roundings are always performed to make the allowed range more strict, making it more safe for the protocol

Issue_23	Spot-price fallback range in <code>rebalance</code> lets liquidity be minted at a manipulated price
Severity	High
Description	For pure-TWAP pairs (no LST rate), <code>_initialTickRange</code> centers the new range on the pool's current tick read from <code>slot0()</code> . Since <code>slot0</code> spot is trivially manipulable within a block, an attacker can move the pool price, trigger a rebalance, and cause the new liquidity to be minted around the manipulated spot. Once the price is restored, that liquidity is at an off-market range and can be swapped for profit. Additionally, this causes a drop in collateral value, enabling opportunistic liquidations, further losing value to MEV
Recommendations	Center the fallback range on the TWAP tick (already computed by <code>_twapTick</code>) rather than raw <code>slot0</code> spot. On top of that, revert if the spot price deviates from fair valuation price beyond a bounded threshold at rebalance time. Do not derive any range or slippage bound from <code>slot0</code> in a state-changing path
Comments / Resolution	Resolved. Rebalance now reverts in case the deviation b/w target price and spot price is greater than allowed deviation. The value set for max deviation will determine the extent to which the manipulation can occur. Note that the roundings involved for the acceptable price range computation doesn't always round for a more strict range, eg: lower end is rounded down rather than up, <code>priceB</code> is rounded down even for the lower end computation. It is ideal that these roundings are always performed to make the allowed range more strict, making it more safe for the protocol

Issue_24	BOT has near-complete control during rebalance and can drain collateral
Severity	High
Description	During rebalance , the BOT supplies swapData that is executed against a whitelisted swap venue via _convertToOptimalRatio , and the only value guards are the caller-supplied minAmount{0,1} and minLiquidity . The rate argument passed into _convertToOptimalRatio is unused for slippage enforcement. A malicious or compromised BOT can therefore convert inventory at an unfavorable ratio, moving value out of the position on every rebalance. This harms depositors directly and, by depressing collateral value, can produce un-liquidatable bad debt that harms lenders as well
Recommendations	Acknowledge and document this finding as part of the trust surface and threat model of the protocol. Alternatively, enforce slippage/value-conservation on-chain using the oracle-derived rate rather than trusting BOT-supplied minAmount/minLiquidity: require the post-swap inventory value [at fair valuation price] to be within a tight tolerance of the pre-swap value. This bounds BOT discretion to “acceptable” execution error and removes the drain vector. Additionally, place limits on how often BOT can rebalance
Comments / Resolution	Resolved in commit 5fbfdc8946c2ff632e4daab9873709629a801d70

Issue_25	Permissionless collectAndCompound compounds idle inventory at manipulated spot price
Severity	High
Description	<code>collectAndCompound</code> is reachable through several routes (it runs inside <code>deposit</code>, <code>withdraw</code>, <code>redeemShares</code>, and is <code>onlyProvider</code> on the adapter, but the provider entry points are permissionless). <code>collectAndCompound</code> adds fees + idle inventory as liquidity via <code>NPM.increaseLiquidity</code> with <code>amount0Min = amount1Min = 0</code>, i.e. at whatever the current spot ratio is, with no slippage protection. When an attacker manipulates the spot price far from the fair valuation price and then triggers compounding, liquidity is added at the manipulated ratio. When the price is restored, value is lost from that liquidity addition and can be captured by the attacker
Recommendations	We recommend that this issue will be fixed. Add slippage protection to compounding: require spot to be within a bounded deviation of the fair/TWAP price before compounding, and/or pass non-zero <code>amount0Min/amount1Min</code> derived from the fair price. Potential side-effects must be considered.
Comments / Resolution	Unresolved. Assets remain as idle ie. doesn't get deposited into the pool as liquidity in case the deviation b/w target price and spot price is greater than allowed deviation. The value set for max deviation will determine the extent to which the manipulation can occur Ideally gate all user facing flows that trigger <code>collectAndCompound</code> behind a trusted role with caller supplied slippage parameters, since the compound currently adds liquidity and relies solely on <code>maxSpotDeviationBps</code> to avoid losses. This is the only solution for CLAMM liquidity additions in integration protocols.

Issue_26	Rebalancing converts impermanent loss into permanent loss (LVR)
Severity	Medium
Description	The system trades a wrapped-native asset against its wrapped-staked version, which has a constant, non-zero price drift (the staking yield / rate appreciation). This continuous drift incurs impermanent loss to LP holders, and the periodic rebalancing (<code>_convertToOptimalRatio</code> + recenter) realizes that impermanent loss as permanent loss each time the range is recentered (<code>rebalance</code>). The net effect is that position value bleeds relative to a "just hold" strategy instead of compounding – a predictable, consistent form of loss-versus-rebalancing (LVR).
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged

Issue_27	Due to LST appreciation, TWAP price consistently lags LST rate
Severity	Medium
Description	TWAP price always lags during price movements. Because the LST is constantly appreciating, the TWAP price is always lower than the LST rate. Valuing deposited collateral at this price will typically result in undervaluing the LST relative to the wrapped native due to appreciation over the averaging period. Newly-deposited liquidity is deposited at the spot price (reflective of LST rate), but valued at the clamped TWAP price (lower). When liquidity is valued at a price away from the current spot, it is overvalued. This results in losses to existing depositors as new entrants can obtain collateral at spot and then deposit it to be composed at the TWAP rate before revaluation at the LST rate. This results in losses to existing depositors as new depositors receive an inflated share of the collateral.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged

Issue_28	TWAP-observe oracle denial-of-service
Severity	Medium
Description	<code>fairSqrtPriceX96</code> reads the pool TWAP via <code>pool.observe([TWAP_PERIOD,0])</code> , which reverts "OLD" whenever the pool's oldest stored observation is younger than <code>TWAP_PERIOD</code> – always true at Uniswap's default <code>observationCardinality</code> of 1, and sustainable by an attacker who compresses the observation buffer when cardinality $N < TWAP_PERIOD/blockTime$. While it reverts, <code>peek</code> reverts, so all pricing, borrowing, and liquidation on those markets are blocked
Recommendations	Call <code>increaseObservationCardinalityNext</code> to ensure that the underlying pools have sufficient cardinality at the time of deployment. Monitor that the observation cardinality is sufficient as blockchain block times decrease.
Comments / Resolution	Acknowledged. Lista will document and operationalize the requirement to allocate the appropriate observation cardinality before deployment

Issue_29	Mismatch between slippage values returned by preview functions and its usage in actual redemption can result in reverts
Severity	Medium
Description	The NPM <code>decreaseLiquidity</code> floor checks only the principal burned from <code>pool.burn</code> ; the pro-rata idle inventory is added to the returned amount afterward, outside the guard. Since the preview i.e. <code>previewRemoveLiquidity/previewRedeemUnderlying</code> , [documented "for tight <code>minAmount{0,1}</code> "] returns principal + idle, a floor sized from the preview reverts whenever idle exceeds the buffer
Recommendations	Check the overall amount received instead of the amount <i>to</i> receive from liquidity removal
Comments / Resolution	Resolved in commit 580e493ff8ef0581a9b2eba23d3a89440d56bb0f

Issue_30	Unenforced rate-derived leg pricing causes overvaluation of collateral
Severity	Low
Description	For the collateral to be valued accurately, the fairPrice used for composition and the valuation price for the underlying(s) should be the same. Although it is highlighted in comments that these two are equivalent, it is not enforced in code anywhere in the reviewed scope. <i>“leg prices are RATE-CONSISTENT with the composition above, NOT an independent, second market price. By deployment invariant, the resilient oracle prices the LST leg (TOKEN0 = slisBNB / wstETH / wbETH) rate-derived from the SAME on-chain exchange rate used for the composition”</i>
Recommendations	Add a check that these are equal or add another transparent mechanism that structurally ensures equivalence
Comments / Resolution	Acknowledged

Issue_31	<code>_twapTick</code> uses checked math deviating from reference implementation
Severity	Low
Description	<code>V3DexAdapter</code> follows the twap tick calculation implementation from the standard <code>OracleLibrary.consult</code> <pre>int56 tickCumulativesDelta = tickCumulatives[1] - tickCumulatives[0];</pre> But since the adapter is compiled under <code>Solidity 0.8.34</code>, the subtraction can potentially underflow and cause a revert while the reference implementation uses compiler version <code>>=0.5.0 <0.8.0</code> and expects a wrap around
Recommendations	Maintain consistency with the reference implementation and use unchecked
Comments / Resolution	Resolved in commit <code>a3e416da5b4b9ddd351248b680a3f18db81a2efb</code>

Issue_32	<code>collectAndCompound</code> can only be triggered by user actions, causing yield drag during idle periods
Severity	Low
Description	The <code>collectAndCompound</code> function is currently invoked only as part of user-initiated actions: <code>deposit</code>, <code>withdraw</code>, and <code>redeemShares</code>. There is no callable path to trigger compounding independently. This causes the pending fees and idle token amounts to sit as dead capital until any of the user action above happens, losing out of earning trading fees.
Recommendations	Consider adding a function to invoke <code>collectAndCompound</code> directly.
Comments / Resolution	Unresolved in commit 4e9699b10c2b85f2257f4154ec763b4ef8f9043b Since the added compound functionality in the provider is permissionless, consider the below suggestion which also applies to previously mentioned high severity finding: Ideally gate all user facing flows that trigger <code>collectAndCompound</code> behind a trusted role with caller supplied slippage parameters, since the compound currently adds liquidity and relies solely on <code>maxSpotDeviationBps</code> to avoid losses. This is the only solution for CLAMM liquidity additions in integration protocols.

Issue_33	Compounded event emits raw toCompound amounts instead of actual amounts used by increaseLiquidity
Severity	Low
Description	The Compounded event emits the raw toCompound0 and toCompound1 as the token amounts rather than the actual amounts that have been deposited as liquidity <pre>emit Compounded(toCompound0, toCompound1, liquidityAdded);</pre>
Recommendations	Consider emitting the actual amounts consumed by increaseLiquidity [i.e. toCompound0 - leftover0 and toCompound1 - leftover1] rather than the raw toCompound0 and toCompound1 values.
Comments / Resolution	Resolved in commit f0d4499eb735c5fffa97d5ceacb9285f91ae007

Issue_34	minAmount floor is only applied for liquidity removal and not on the overall amount
Severity	Low
Description	The caller's minAmount0/minAmount1 slippage floor is forwarded only to decreaseLiquidity and not applied on the overall amount received which includes idle token amounts. The check runs solely inside if [liquidityToRemove > 0], so on an idle-only / zero-liquidity exit (or when the pro-rata liquidityToRemove = totalLiq-shares/totalShares rounds to 0 on a dust-share redemption) and no floor is even enforced
Recommendations	Check the overall amount received rather than the amount to receive from liquidity removal
Comments / Resolution	Resolved in commit 580e493ff8ef0581a9b2eba23d3a89440d56bb0f

Issue_35	<code>fairSqrtPriceX96</code> trusts TWAP without a spot-deviation guard
Severity	Low
Description	For non-rate pairs, <code>fairSqrtPriceX96</code> returns the TWAP-derived price and this price is used to compute the current token composition of the position. If the actual price differs substantially from the TWAP due to sudden fluctuations, the composition (and every valuation derived from it) is computed at an incorrect price. A safer design reverts on large TWAP-vs-spot deviation rather than silently trusting a stale price
Recommendations	Ideally use the oracle prices rather than using TWAP
Comments / Resolution	Acknowledged. Lista will not deploy pairs without an independent non-TWAP oracle

Issue_36	<code>_initialTickRange</code> and <code>_tickRangeForRate</code> can produce out-of-bounds ticks near tick extremes
Severity	Low
Description	The <code>_initialTickRange</code> function doesn't clamp the result to the valid tick range [<code>TickMath.MIN_TICK</code> to <code>TickMath.MAX_TICK</code>]. Hence if the pool's current tick is close to the ticks of <code>MIN_TICK</code> or <code>MAX_TICK</code>, the subtraction or addition produces a tick beyond the valid boundary. In case this happens, the rebalance function will revert
Recommendations	Acknowledge or clamp <code>initialTickLower</code> and <code>initialTickUpper</code> to [<code>MIN_TICK</code> , <code>MAX_TICK</code>] after the <code>_floorTick</code> / <code>_ceilTick</code> alignment, ensuring the computed range is always valid regardless of the pool's current tick or the rate-derived center
Comments / Resolution	Flawed fix. The following tick adjustment is incorrect when the condition becomes true due to <code>initialTickLower</code> being increased to <code>minUsable</code> <pre> if (initialTickLower >= initialTickUpper) { initialTickLower = maxUsable - tickSpacing; initialTickUpper = maxUsable; } </pre>

Issue_37	<code>rebalance</code> recenters even when the new ticks equal the old ticks
Severity	Informational
Description	<code>rebalance</code> performs a full remove/convert/mint cycle even when <code>_initialTickRange</code> yields ticks identical to the current <code>tickLower/tickUpper</code> , wasting gas and unnecessarily realizing fees/rounding when nothing changes
Recommendations	Acknowledge this finding. Alternatively, short-circuit when the computed range equals the current range [optionally still compounding], avoiding an unnecessary burn/mint cycle.
Comments / Resolution	Resolved in commit <code>f0d4499eb735c5ffffa97d5ceacb9285f91ae007</code> . But the conditions under which the re-balance is short circuited is abnormal and requires the critical parameters <code>minAmounts</code> , <code>minLiquidity</code> and <code>targetSqrtPriceX96</code> to be 0 which could be loss-inflicting when a rebalance actually occurs (in case the ticks end up being different)

Issue_38	Rebalance deviation threshold can expose a permanently-idle price band
Severity	Informational
Description	The <code>centerRateThresholdBps</code> deviation required to allow a rebalance should not be larger than the range half-width, otherwise there is a band of prices in which the position is out of range (idle) but a rebalance is not yet permitted. Ideally the threshold is <code><= INITIAL_RANGE_BPS</code>
Recommendations	Acknowledge and document this finding
Comments / Resolution	Acknowledged

Issue_39	Reentrant swap pair during rebalance can zero out slisBNBx balances
Severity	Informational
Description	If a whitelisted swap pair allows reentrancy during rebalance , a reentrant read of getUserBalanceInBnb would return 0 because at that moment there is no underlying V3 position (tokenId == 0 mid-rebalance), which could drop users' slisBNBx balances to zero.
Recommendations	Acknowledge and document this finding. Operationally, ensure that only non-reentrant swap venues are whitelisted
Comments / Resolution	Acknowledged

Issue_40	previewRemoveLiquidity doesn't include pending outstanding fees
Severity	Informational
Description	The previewRemoveLiquidity function doesn't include the outstanding fees. While during actual withdrawal, the collectAndCompound function will be collecting and adding the fees to the liquidity position. This will cause the returned amounts from previewRemoveLiquidity to be understated when compared with actual withdrawal
Recommendations	Acknowledge this finding. Alternatively, simulate the collectAndCompound function inside previewRemoveLiquidity
Comments / Resolution	Resolved in commit <code>dac257d8d5c5bc4ce34f9a029604d14aabc9170f</code> . Note that slight inconsistency is still possible since the pending fees/idle token amounts might be added as liquidity before the actual withdraw happens

Issue_41	<code>_tickAtSqrtRatio</code> reimplements <code>TickMath.getTickAtSqrtRatio</code> with a binary search
Severity	Informational
Description	<code>_tickAtSqrtRatio</code> inverts <code>getSqrtRatioAtTick</code> via a ~22-iteration binary search to recover the tick from a sqrt price. Uniswap's <code>TickMath</code> library already provides this as a closed-form <code>getTickAtSqrtRatio</code> , so the loop is redundant and much more gas-costly. Lastly, <code>getTickAtSqrtRatio</code> reverts on inputs outside <code>[MIN_SQRT_RATIO, MAX_SQRT_RATIO]</code> , whereas the binary search silently clamps to <code>MIN_TICK/MAX_TICK</code> and returns a tick that doesn't satisfy the <code>getSqrtRatioAtTick(tick) <= sqrtPriceX96</code> invariant.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged

SlisBNBV3DexAdapter

The SlisBNBV3DexAdapter contract is the slisBNB/WBNB implementation of **V3DexAdapter** on BNB Chain. The contract restricts its configuration to the slisBNB/WBNB pair and uses WBNB as the wrapped-native token.

The fair price and position-center rate are derived from the slisBNB-to-BNB conversion rate reported by Lista's StakeManager rather than directly from the pool's spot price or TWAP. The initial position and subsequent rate-based rebalances use a narrow range centered around this conversion rate.

All NFT management, fee compounding, withdrawal, idle-inventory accounting, and rebalance swap execution are inherited from the base adapter. Inventory conversion during rebalancing is performed through backend-generated calldata sent to a manager-approved swap venue

Appendix: slisBNB/BNB Rate-Centered Position

The **slisBNB** adapter fixes the strategy pair to **slisBNB/WBNB** and derives its valuation and range center from Lista's **StakeManager** rather than from the pool spot price:

```
> centerRate = STAKE_MANAGER.convertSnBnbToBnb[1e18]
```

At initialization, this rate is used to calculate a tick-spacing-aligned range approximately bounded by:

```
> lowerRate = centerRate * 99%
```

```
> upperRate = centerRate * 101%
```

The same **StakeManager** rate is returned through the base adapter's fair-price mechanism and is recorded as **lastCenterRate**. By default, a new rebalance is permitted after the rate has moved by at least the configured 1% center-rate threshold.

The pool may still determine the token amounts consumed when liquidity is minted, but raw pool spot and TWAP are not used as the fair valuation anchor for the slisBNB position.

Privileged Functions

- none

Invariants

INV 1: **TOKEN0** must be **slisBNB**, **TOKEN1** must be **WBNB**, and the wrapped-native token must be **WBNB**

INV 2: The center rate used for rebalancing must be derived from the same **StakeManager** conversion relationship used for fair valuation

Issue_42	SlisBNBV3DexAdapter._isSlisBnbWbnbPool always returns the same value
Severity	Informational
Description	SlisBNBV3DexAdapter._isSlisBnbWbnbPool appears to always evaluate to true for the intended deployment, so the branch it guards is effectively constant and adds unnecessary complexity
Recommendations	If the pool is fixed at deployment, simplify the implementation by removing the runtime check (or return true) and validate the pool composition once in the constructor
Comments / Resolution	Resolved in commit 791bc490202f12a5498a779baa414eef1acd8afb

WbETHV3DexAdapter

The WbETHV3DexAdapter contract is the wbETH/WETH implementation of **V3DexAdapter** on Ethereum. It restricts the managed pair to wbETH and WETH and uses the exchange rate reported by the wbETH contract as the underlying wbETH-to-ETH rate.

The exchange rate is used to center the concentrated-liquidity range and to establish the permitted bounds for the valuation price. The adapter normally uses the pool TWAP to determine the position's token composition, but clamps that TWAP within a configurable deviation band around the wbETH exchange rate. If the deviation is configured as zero, the exchange rate itself is used without reading the pool TWAP.

The contract inherits the base NFT custody, liquidity management, compounding, idle-inventory tracking, and whitelisted rebalance swap mechanism. The manager may configure the TWAP deviation band, subject to the contract's maximum permitted deviation

Appendix: wbETH Exchange-Rate and TWAP Clamp

The **wbETH** adapter derives its fundamental **wbETH/WETH** rate from:

```
> rate = WBETH.exchangeRate()
```

The liquidity range is centered around this rate with an approximate $\pm 1\%$ width. Position valuation may additionally reflect the pool TWAP, but only inside a manager-configured deviation band.

For a deviation value **d**:

```
> lowerBound = rate * (10,000 - d) / 10,000  
> upperBound = rate * (10,000 + d) / 10,000  
> fairPrice = clamp(poolTwap, lowerBound, upperBound)
```

When **maxTwapDeviationBps** is zero, the TWAP read is skipped and the fair price equals the **wbETH** exchange rate. The initial deviation is 1%, and the manager may configure it only up to the hard maximum of 10%.

The range center continues to follow **WBETH.exchangeRate()** even when the valuation composition is taken from a TWAP inside the permitted band.

Privileged Functions

- `setMaxTwapDeviationBps`

Invariants

INV 1: `TOKEN0` must be `wbETH`, `TOKEN1` must be `WETH`, and the wrapped-native token must be `WETH`

INV 2: The center rate used for a rebalance must remain consistent with the `wbETH` exchange rate

Issue_43	<code>setMaxTwapDeviationBps</code> changes share pricing mid-flight, affecting V3Provider deposit accounting
Severity	Medium
Description	The <code>setMaxTwapDeviationBps</code> function in <code>WbETHV3DexAdapter</code> and <code>WstETHV3DexAdapter</code> updates <code>maxTwapDeviationBps</code> in storage with immediate effect. This can cause the <code>fairSqrtPriceX96()</code> function to have a discrete jump in returned value without any change to underlying prices since the clamp band changes. This can cause sudden changes in the collateral price (possibly enabling liquidations) and also on the amount of shares received for a deposit
Recommendations	Acknowledge this finding and document this behaviour
Comments / Resolution	Acknowledged

WstETHV3DexAdapter

The WstETHV3DexAdapter contract is the wstETH/WETH implementation of **V3DexAdapter** on Ethereum. It restricts the managed position to the wstETH/WETH pair and derives the underlying wstETH-to-ETH conversion rate from Lido's **stEthPerToken()** value.

The conversion rate is used to center the liquidity range and define the acceptable boundaries for the position's valuation price. Under the default mechanism, the pool TWAP determines the position's token composition while being clamped within a configurable deviation band around the Lido conversion rate. A zero deviation setting causes the adapter to use the conversion rate directly and removes the dependency on the pool's TWAP observations.

The remaining NFT management, liquidity addition and removal, compounding, idle-balance accounting, and inventory-conversion logic are inherited from **V3DexAdapter**. Rebalances may only be initiated through the configured provider and may interact only with manager-approved swap venues

Appendix: wstETH Conversion Rate and TWAP Clamp

The wstETH adapter derives the rate used for range centering from Lido's accounting conversion:

```
> rate = WSTETH.stEthPerToken()
```

The strategy's tick range is centered around this rate with an approximate $\pm 1\%$ width. For valuation, the adapter uses the pool TWAP only within a bounded region around the conversion rate.

For a configured deviation d :

```
> lowerBound = rate * (10,000 - d) / 10,000  
> upperBound = rate * (10,000 + d) / 10,000  
> fairPrice = clamp(poolTwap, lowerBound, upperBound)
```

A zero deviation causes the contract to use the Lido conversion rate directly and avoids any dependency on V3 observation history. The default deviation is 1%, while the manager-configurable hard maximum is 10%.

The exchange rate is also stored as the rebalance center, allowing the position to be recentered as staking yield causes the wstETH/WETH relationship to appreciate over time.

Privileged Functions

- `setMaxTwapDeviationBps`

Invariants

INV 1: `TOKEN0` must be `wstETH`, `TOKEN1` must be `WETH`, and the wrapped-native token must be `WETH`

INV 2: The underlying `wstETH/WETH` rate must equal `WSTETH.stEthPerToken()`

INV 3: The center rate used for a rebalance must remain consistent with the `wstETH` conversion rate

Issue_44	<code>_twapTick</code> reverts on fresh or low-cardinality pools, bricking deposits and health checks
Severity	Low
Description	The <code>_twapTick</code> function assumes that observations old enough for <code>TWAP_PERIOD</code> will be available. But in case the observation cardinality has not been expanded sufficiently the oldest observation can be younger than <code>TWAP_PERIOD</code> (if the cardinality is low, attacker can make this happen by calling <code>observe</code> in regular timestamps). This will cause the <code>fairSqrtPriceX96</code> function to revert
Recommendations	Call <code>increaseObservationCardinalityNext</code> to ensure that the underlying pools have sufficient cardinality at the time of deployment. Monitor that the observation cardinality is sufficient as blockchain block times decrease
Comments / Resolution	Acknowledged

SwapInventoryLib

The SwapInventoryLib library is a DEX-agnostic helper which performs the inventory-conversion swap used by the V3 adapters during a rebalance. The library does not determine a route or calculate an expected market price; instead, it forwards backend-generated calldata to the swap venue already approved by the adapter.

The library measures the adapter's token balances before and after the external call to determine the actual amount spent and received. It ensures that the swap does not consume more than the available or requested input amount and that the received output satisfies the supplied minimum-output requirement.

The library supports both ERC-20 and native-currency swap conventions. A wrapped-native input may be unwrapped and sent as native currency, while native currency returned by the swap is wrapped back into the corresponding ERC-20 so that it remains included in the adapter's inventory accounting. Temporary ERC-20 approvals are removed after the external swap call

Appendix: Inventory-Conversion Swap Flow

The library performs one optional token conversion during a strategy rebalance. The route and calldata are constructed off-chain, while the library bounds the external execution through measured balance changes.

The swap flow is:

1. Select `tokenIn` and `tokenOut` from `sellToken0`.
2. Cap `amountIn` to the inventory amount recorded for the selected input leg.
3. Record the adapter's input-token, output-token, and native-currency balances.
4. For an ERC-20 input, approve only `amountIn` to the swap venue; for a native input, unwrap the wrapped-native token and forward the amount as `msg.value`.
5. Execute the supplied calldata against the whitelisted venue.
6. Wrap any native currency returned by the venue back into the configured wrapped-native ERC-20.
7. Calculate the actual input spent and output received from the balance deltas.
8. Require `spent <= amountIn` and `received >= amountOutMin`.
9. Update the in-memory strategy inventory by subtracting the measured input and adding the measured output.

For ERC-20 swaps, the temporary venue allowance is cleared after execution. The output must be delivered to the adapter because the library executes through `DELEGATECALL` and measures balances on the adapter itself.

Privileged Functions

- none

Invariants

INV 1: A token swap must never spend more input tokens than the amount explicitly supplied to the library

INV 2: A swap must revert when the measured output is below the caller's minimum-output requirement

INV 3: Token approval granted to a swap spender must be limited to the current swap amount and cleared after execution

INV 4: A swap must not allow the target or spender to consume unrelated token or native-currency balances

Issue_45	<code>SwapInventoryLib.swap</code> reverts if whitelisted swap pair refunds native coin to the adapter
Severity	Medium
Description	The <code>SwapInventoryLib.swap</code> function expects the native token balance to remain the same unless either of the tokens involved in the swap is wrapped native. But this doesn't always need to hold true. For eg: Velodrome's SlipStreamRouter's <code>exactInput</code> forwards any native coin balance available in the address. Any attacker can donate 1 wei of native token to the router in order to cause revert
Recommendations	Check the native token balance increase and deposit to wrapped native only when either of the tokens is wrapped native
Comments / Resolution	Resolved in commit 167ece5713ebd6d1a45dfb5a8221306cd4bd31dd

Issue_46	Capped swap allowance can revert rebalance
Severity	Low
Description	<code>SwapInventoryLib.swap</code> caps <code>amountIn</code> to the position's available balance <code>avail</code> and sets the venue ERC-20 allowance to that capped value, but forwards the BOT-built <code>swapData</code> unchanged. That opaque calldata encodes the pre-cap <code>amountIn</code> , so if the pool moved between the BOT's off-chain simulation and on-chain execution such that <code>avail</code> is smaller than assumed, the logic can break. For example, the venue's <code>transferFrom</code> for the larger encoded amount can exceed the capped allowance reverting the entire <code>rebalance</code>
Recommendations	Ensure that there is some mechanism to read the new input amount at the <code>swapPair</code>
Comments / Resolution	Resolved. Since the function now reverts in case <code>avail</code> is $< \text{amountIn}$, it can cause <code>rebalance</code> to revert if share redemptions occur before the <code>rebalance</code> tx execution

Issue_47	<code>SwapInventoryLib</code> empty-leg no-op skips <code>amountOutMin</code>
Severity	Low
Description	<code>SwapInventoryLib.swap</code> caps <code>amountIn</code> to the available leg (<code>avail</code>) and early-returns (<code>total0</code> , <code>total1</code>) unchanged when <code>amountIn == 0</code> or <code>avail == 0</code> , <i>before</i> the <code>received >= amountOutMin</code> assertion, so a rebalance intending to convert a side that turns out empty silently recenters without converting and never asserts the intended output floor
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged

V3PositionLib

The V3PositionLib library is a helper library which contains the low-level interactions with the V3 **NonfungiblePositionManager**. It is used by the adapter when minting a position, increasing or decreasing liquidity, collecting tokens, and burning an empty position NFT

The library prepares the position-manager parameter structures, grants the required token allowances, and configures the calling adapter as the recipient of newly minted NFTs and collected tokens. Liquidity removal and token collection are kept as separate operations because decreasing liquidity records the corresponding assets as tokens owed to the NFT before they can be collected.

Appendix: Nonfungible Position Manager Lifecycle

V3PositionLib contains the external-library primitives used to manage the strategy NFT. Because the functions execute through **DELEGATECALL**, token balances, approvals, NFT ownership, and collection recipients resolve to the calling adapter.

The position lifecycle is divided into the following operations:

- **Mint**: Approve the desired token amounts and mint a new V3 NFT to **address[this]** using the supplied pair, fee tier, ticks, minimum amounts, and the current block timestamp as the deadline.
- **Increase liquidity**: Approve the desired amounts and add liquidity to an existing **tokenId**, returning the exact liquidity and token amounts consumed by the position manager.
- **Decrease liquidity**: Remove a specified liquidity amount; the removed principal is first recorded by the position manager as tokens owed to the NFT.
- **Collect**: Transfer all currently owed **token0** and **token1** amounts, including collected fees and removed principal, to **address[this]**.
- **Burn**: Destroy an NFT after its liquidity has been removed and its collectible balances have been cleared.

The separation between **decreaseLiquidity** and **collectAll** is important because decreasing liquidity does not itself transfer the underlying tokens to the adapter.



Privileged Functions

None

Invariants

INV 1: Liquidity and token-amount conversions must use the NFT's actual token pair, tick range, and current liquidity

INV 2: Calculating position amounts at a price must return the canonical Uniswap V3 amounts for that liquidity and tick range

INV 3: Increasing liquidity must return the exact liquidity added and the exact token amounts consumed by the position manager

INV 4: A position NFT may only be burned after its liquidity is zero and all collectible token amounts have been collected

Issue_48	One sided idle inventory reverts <code>collectAndCompound</code> , DoSing user flows
Severity	High
Description	When idle collateral is one-sided, every attempt to <code>collectAndCompound()</code> will revert because zero <code>liquidity</code> is added. This DoSes <code>deposit</code> , <code>withdraw</code> , <code>redeemShares</code> , and <code>withdrawShares/supplyShares</code> once the related issues are resolved
Recommendations	Add an explicit check for zero computed liquidity to <code>_collectAndCompound</code> . Make it a no-op (return) when the computed addable liquidity is zero
Comments / Resolution	Resolved in commit <code>af0bff2272a787b519d4c5ab2a84875f8f34c76e</code>

Issue_49	Excess NPM allowance is not cleared after <code>mint</code>
Severity	Low
Description	<code>V3PositionLib.mint</code> calls <code>safeIncreaseAllowance</code> for both tokens by <code>amountXDesired</code> before minting, but when the NPM does not consume the full desired amount the residual allowance is left standing on the NPM. Over many operations this accumulates a non-zero allowance to the position manager
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged

V3ProviderOracle

The V3ProviderOracle contract is a standalone oracle which calculates the value of a **V3Provider** share for use as collateral inside Moolah. It prices the complete managed position rather than relying on the V3 pool's raw spot price.

The oracle obtains the position's token composition from the configured adapter at its fair price, values both token legs through the resilient oracle, and divides the resulting value by the provider's total share supply. The result represents the price of one whole provider share and is reduced by a configurable collateral haircut.

When there are no outstanding shares, the provider-share price is returned as zero. If shares exist but an underlying token price or the total position value is zero, the call reverts so that Moolah does not rely on an invalid collateral price. Queries for tokens other than the configured provider share are delegated directly to the resilient oracle.

Appendix: Provider-Share Price Calculation

The oracle prices the complete managed position at the adapter's fair token composition. For a non-zero provider share supply, it performs the following calculation:

1. Read `[total0, total1]` from `ADAPTER.positionAmountsAt(ADAPTER.fairSqrtPriceX96())`
2. Read the resilient-oracle USD prices of both token legs.
3. Normalize each token amount by its token decimals and calculate the total 8-decimal USD value:

```
> totalValueUsd = total0 * price0 / 10^decimals0 + total1 * price1 / 10^decimals1
```

4. Convert the total value into the price of one whole provider share:

```
> rawSharePrice = totalValueUsd * 10^shareDecimals / totalSupply
```

5. Apply the configured collateral haircut:

```
> providerSharePrice = rawSharePrice * (10,000 - haircutBps) / 10,000
```

The haircut is capped at 10% and affects only the **Moolah** collateral price; it does not remove assets from the provider or alter the amount received when shares are redeemed.

Appendix: Failure and Delegation Behaviour

If the provider has no outstanding shares, the provider-share price is returned as zero because no live collateral unit exists. Once shares are outstanding, a zero price for either token leg or a zero total position value causes the oracle to revert rather than return an apparently valid zero-valued collateral price.

For every token other than the configured provider share, both `peek` and `getTokenConfig` are delegated directly to the resilient oracle. The contract therefore acts as a specialized wrapper only for the V3 provider share while preserving the underlying oracle's behavior for all other assets.

Privileged Functions

- `_authorizeUpgrade`
- `setHaircutBps`

Invariants

INV 1: The provider-share price must be based on `adapter.positionAmountsAt(adapter.fairSqrtPriceX96())`

INV 2: The returned provider-share price must equal the raw share price reduced by the configured haircut

INV 3: When provider `totalSupply` is zero, the provider-share oracle price must be zero.

INV 4: `haircutBps` must remain between zero and `MAX_HAIRCUT_BPS`

INV 5: For tokens other than the provider share, `peek` and token-configuration queries must delegate unchanged to the resilient oracle

Issue_50	Staked-token depeg/solvency risk is excluded from the risk model, creating un-liquidatable bad debt
Severity	High
Description	The valuation deliberately prices the LST leg from the on-chain redemption rate rather than a secondary market price. <code>V3ProviderOracle.peek</code> documents that <code>peek[LST] == peek[underlying] × exchangeRate / 1e18</code> and the adapter's <code>fairSqrtPriceX96</code> is rate-implied [or a TWAP clamped to the rate]. This means a real depeg or solvency event in the underlying staked token (slisBNB/stETH/wbETH) is invisible to the collateral price: positions that are actually insolvent remain reported as healthy, so they cannot be liquidated and the resulting bad debt is stuck. The bounded-divergence / clamped-TWAP oracle is the worst of both worlds here – it neither tracks the market during a depeg nor protects against manipulation better than a plain rate
Recommendations	Incorporate the LST market price into the risk model – e.g. price the LST leg at <code>min(rate-implied, market)</code> for collateral valuation, or add a depeg circuit breaker that switches the market to liquidation-only / lowers effective LLTV when the market price diverges from the redemption rate beyond a threshold. Treat the redemption rate as an upper bound, not the truth, for collateral
Comments / Resolution	Acknowledged with the comment "This is a known product-risk assumption. Only robust assets are onboarded, with depeg monitoring and operational liquidation response." It must be considered that the LST rate could remain the same even in the event of a depeg

Issue_51	Haircut seizure vs unhaircut redemption over-penalizes borrower
Severity	Medium
Description	A lower [haircut] collateral price makes seizedAssets larger [seizure is inverse to price]; those extra shares then redeem at un-haircut value, so the effective liquidation penalty to the borrower is amplified by the haircut beyond the intended liquidation bonus. This is the standard consequence of a discounted collateral oracle paired with un-discounted redemption – a real borrower disadvantage distinct from haircut-as-lending-buffer
Recommendations	Account for the extra risk in LLTV rather than using haircut
Comments / Resolution	Acknowledged

Issue_52	<code>V3ProviderOracle.peek</code> reverts for small positions, blocking liquidations and accruing bad debt
Severity	Low
Description	The <code>V3ProviderOracle.peek</code> function can produce <code>totalValue == 0</code> when the adapter's position holds a very small amount of liquidity due to both terms rounding down to zero <pre>uint256 totalValue = (total0 * price0) / (10 ** DECIMALS0) + (total1 * price1) / (10 ** DECIMALS1);</pre> When <code>totalValue</code> is 0 the function reverts which upstream causes the <code>Moolah.liquidate</code> call to revert hence blocking liquidations. Although under normal conditions initiating a loan with dust collateral values isn't possible due to the <code>minLoan</code> guard, it is possible for the collateral value to drop to 0 in the event of an extreme price crash. If this happens, the borrow position backed by the small amount of <code>V3Provider</code> shares becomes unliquidatable and continues accruing interest and generating bad debt
Recommendations	Consider an alternative and conservative floor pricing when attempting to liquidate a bad debt position instead of reverting the oracle entirely which can revert the whole liquidation call
Comments / Resolution	Resolved in commit 3a4cbeac02703aca18db58a90f57487a8f884a0e

Issue_53	<code>setHaircutBps</code> applies instantly with no rate-limiting or cooldown
Severity	Low
Description	The <code>setHaircutBps</code> function updates the <code>haircutBps</code> with immediate effect. This will cause the price of the collateral/share token to immediately change which can cause liquidations of previously-healthy positions
Recommendations	Acknowledge this finding and document this behaviour. Alternatively, implement a timelock or smoothing function for the variable update
Comments / Resolution	Acknowledged

Issue_54	<code>setHaircutBps</code> emits an event without confirming the value changed
Severity	Informational
Description	<code>setHaircutBps</code> emits <code>HaircutChanged</code> without checking that the new value differs from the current one, producing misleading no-op events.
Recommendations	Acknowledge this finding. Alternatively, skip the write/emit (or revert) when <code>_haircutBps == haircutBps</code> .
Comments / Resolution	Resolved in commit 791bc490202f12a5498a779baa414eef1acd8afb

Issue_55	V3ProviderOracle.initialize does not emit <code>HaircutChanged</code>
Severity	informational
Description	<code>initialize</code> sets <code>haircutBps</code> but does not emit <code>HaircutChanged</code> , so off-chain indexers cannot observe the initial haircut from events alone
Recommendations	Acknowledge this finding. Alternatively, emit <code>HaircutChanged[_haircutBps]</code> in <code>initialize</code>
Comments / Resolution	Resolved in commit 791bc490202f12a5498a779baa414eef1acd8afb

V3Liquidator

The V3Liquidator contract is a role-controlled liquidation helper for Moolah markets whose collateral token is a **V3Provider** share. It supports both pre-funded liquidations and callback-based flash liquidations.

In a pre-funded liquidation, the contract uses loan tokens already held by the liquidator contract to repay the borrower's debt and retains the seized provider shares. In a flash liquidation, Moolah first transfers the seized shares to the contract and invokes its callback. The callback can redeem those shares into the two underlying position tokens, swap both of the tokens into the market's loan token, and approve Moolah to collect the required repayment within the same transaction.

The contract also exposes **BOT**-controlled functions for redeeming provider shares and swapping token or native-currency balances outside the liquidation callback. Markets, providers, tokens, swap targets, and separate approval spenders are controlled through manager-maintained whitelists, while the manager may withdraw residual assets held by the contract

Appendix: Pre-Funded Liquidation Flow

The basic **liquidate** function assumes that the liquidator contract already holds enough of the market's loan token to cover Moolah's required repayment.

1. A **BOT** selects a manager-whitelisted market and the borrower to liquidate.
2. The liquidator grants **Moolah** an allowance over the market loan token.
3. **Moolah** reduces the borrower's debt, transfers the seized provider-share collateral to the liquidator, and pulls the repayment amount.
4. The loan-token allowance is cleared after the liquidation.
5. The seized provider shares remain in the liquidator and may later be redeemed through **redeemV3Shares**.

This mode separates liquidation from collateral conversion and therefore requires the contract to be funded before execution.

Appendix: Flash Liquidation Flow

The flash path converts seized provider shares into the loan token inside Moolah's liquidation callback:

1. A BOT calls `flashLiquidate` using a whitelisted market and V3 provider; the provider must be the market's configured collateral token.
2. Moolah calculates the liquidation amounts, transfers the seized provider shares to the liquidator, and calls `onMoolahLiquidate` before collecting repayment.
3. If `redeemShares` is enabled, the callback burns the seized shares through the provider and receives both underlying legs; the wrapped-native leg is delivered as native currency.
4. Each redeemed leg may be swapped into the loan token through a whitelisted pair and, where applicable, a separately whitelisted approval spender.
5. If the loan token is the wrapped-native asset, the callback wraps the available native balance back into the ERC-20 representation.
6. The callback verifies that the contract holds at least `repaidAssets` of the loan token and approves exactly that amount to Moolah.
7. After the callback returns, Moolah pulls the approved repayment and finalizes the borrower's debt and collateral changes.

When `redeemShares` is disabled, the seized shares remain in the liquidator and the repayment must be covered by loan tokens already held by the contract.

Appendix: Standalone Redemption and Swap Operations

A BOT may separately redeem whitelisted provider shares held by the liquidator or convert residual token balances through `sellToken` and `sellBNB`.

For standalone swaps, the contract records pre-swap balances and enforces:

```
> actualInput = balanceInBefore - balanceInAfter
> actualOutput = balanceOutAfter - balanceOutBefore
> actualInput <= amountIn
> actualOutput >= amountOutMin
```

The token input, output, pair, and any separate spender must be present in the manager-controlled whitelists. ERC-20 approvals are cleared after the external call, while residual assets can only be withdrawn by the `MANAGER` role.

Privileged Functions

- `_authorizeUpgrade`
- `withdrawERC20`
- `withdrawETH`
- `setTokenWhitelist`
- `setMarketWhitelist`
- `batchSetMarketWhitelist`
- `setPairWhitelist`
- `setV3ProviderWhitelist`
- `batchSetV3Providers`
- `liquidate`
- `flashLiquidate`
- `redeemV3Shares`
- `sellToken`
- `sellBNB`
- `onMoolahLiquidate`

Invariants

INV 1: A liquidation may only be initiated by an account holding the **BOT** role.

INV 2: A liquidation may only operate on a whitelisted **Moolah** market

INV 3: A flash liquidation may only use a whitelisted V3 provider

INV 4: `onMoolahLiquidate` may only be called by Moolah

INV 5: The callback must never redeem more provider shares than the liquidator actually received from **Moolah**

INV 6: The underlying assets received from redemption must equal the seized shares' pro-rata entitlement

INV 7: Before callback completion, the liquidator must hold and approve enough loan tokens for **Moolah** to collect the required repayment

INV 8: Every liquidation swap target and separate spender must be whitelisted

INV 9: Swap approvals must be limited to the current swap and cleared after execution.

INV 10: For a token swap, actual input must equal the input-token balance decrease and actual output must equal the output-token balance increase

INV 11: Only the **MANAGER** role may withdraw assets or modify token, pair, market, and provider whitelists

INV 12: At the end of a successful liquidation, **Moolah's** debt and collateral changes must reconcile exactly with the repayment and seized collateral transferred through the liquidator.

Issue_56	Dusting prevents bad-debt socialization and liquidation of underwater positions
Severity	Medium
Description	Moolah only performs bad-debt socializations when <code>position[id][borrower].collateral == 0</code> , exactly. An attacker can frontrun a liquidation attempt with dusting the borrower's collateral by abusing <code>onBehalf</code> to cause <code>collateral</code> to be 1 wei above the <code>seizedAssets</code> . For an underwater position, this causes <code>_isHealthyAfterLiquidate</code> to fall through to <code>_isHealthy</code> , which returns <code>false</code> because the position is now below the <code>minLoan</code> threshold, reverting the liquidation attempt
Recommendations	There are some options: • Enable bad debt socialization when <code>collateral</code> is below some dust threshold (potentially <code>minLoan</code> itself) • Skip <code>minLoan</code> checking during liquidation • Disallow the use of or remove the argument <code>onBehalf</code> in <code>deposit</code> and <code>supplyShares</code> • Establish economically relevant minimums for <code>supplyCollateral</code>
Comments / Resolution	Unresolved. Client acknowledged with the comment "Only robust assets are onboarded, depeg monitoring is in place, and liquidation responsiveness is treated as an operational prerequisite.". This issue isn't covered by any of the operational procedures mentioned above

Issue_57	<code>batchSetV3Providers</code> lacks the same-status check of the single-element setter
Severity	Low
Description	<code>setV3ProviderWhitelist</code> reverts with <code>WhitelistSameStatus()</code> when the status is unchanged, but the batch variant <code>batchSetV3Providers</code> performs no such check and unconditionally writes and emits. This allows redundant writes and misleading events
Recommendations	Acknowledge this finding
Comments / Resolution	Acknowledged

Issue_58	<code>onMoolahLiquidate</code> profitability check can be satisfied by pre-existing tokens
Severity	Low
Description	The final check in <code>V3Liquidator.onMoolahLiquidate</code> does not guarantee the liquidation was actually profitable, because tokens that already existed in the liquidator (pre-existing balances) can be counted toward satisfying the check. A liquidation that is net-unprofitable can appear to pass.
Recommendations	Measure profitability by the delta of balances across the liquidation (<code>balanceOf</code> -after minus <code>balanceOf</code> -before) rather than absolute balances, so pre-existing tokens cannot mask an unprofitable liquidation
Comments / Resolution	Resolved

Issue_59	<code>V3Liquidator._swapRedeemedLeg</code> sends native to the pair rather than the spender
Severity	Low
Description	In <code>V3Liquidator._swapRedeemedLeg</code> , when the token is the wrapped-native asset, the native token is sent directly to the pair rather than to the spender. This is an inconsistency in the settlement path that may not match the router/pair's expected funding source
Recommendations	Acknowledge and document this finding
Comments / Resolution	Acknowledged